

Err and err and err again, but less and less and less.

Aula 12

Linguagens Independentes de Contexto (CFL)

Já sabemos que há linguagens que não são regulares. Que podemos fazer aos autómatos (DFA e NFA) por forma a que estes passem a ser mais expressivos, ou seja reconheçam mais linguagens?

Se munirmos os autómatos de memória genérica adicional ficamos imediatamente com uma expressividade que é muito superior à que desejamos.

A solução é a de dotar os autómatos de uma estrutura de pilha (um *stack*).

Uma pilha (que em geral não tem profundidade máxima) podemos fazer somente três operações:

- **TOP**(ℓ) que verifica se o a célula do topo da pilha contém o símbolo ℓ . O resultado é, portanto, booleano;
- **POP** que apaga a célula de topo da pilha;
- **PUSH**(x) que coloca o símbolo x no topo da pilha, tendo previamente “empurrado” todas as células desta para a posição imediatamente mais abaixo do que a que estava.

Autómatos de pilha (PDA)

Definição

Um autómato de pilha (PDA) A é um tuplo ordenado

$$\langle Q, \Sigma, \Gamma, \delta, I, z_0, F \rangle,$$

com

- Q é o conjunto (finito) de estados do autómato,
- Σ é o alfabeto da linguagem de input,
- Γ é o alfabeto usado na pilha,
- δ é a função de transições, com

$$\delta : Q \times (\Sigma \cup \{\varepsilon\}) \times (\Gamma \cup \{\varepsilon\}) \mapsto 2^{Q \times \Gamma^*}$$

- $q_0 \subseteq Q$ é o estado inicial,
- $z_0 \in \Gamma$ é o símbolo inicial que reside sozinho, inicialmente, na pilha,

- $F \subseteq Q$ é o conjunto dos estados finais.

Em cada momento, o autómato procede, supondo que se encontra no estado q , e se

$$\delta(q, \sigma, \gamma) \ni (q', w),$$

com $q, q' \in Q, \sigma \in \Sigma, \gamma \in \Gamma, w \in \Gamma^*$, se o caracter que se encontra sob a cabeça de input do autómato é σ e o símbolo que se lê na posição de topo da pilha é γ ,

- move a cabeça de leitura de input uma posição para a direita;
- muda para o estado q' ;
- apaga a célula de topo da pilha (POP);
- coloca, por ordem, cada um dos símbolos de w (PUSH), forçando que cada uma das células já presentes na pilha sejam movidas para a posição imediatamente seguinte.

Se

$$\delta(q, \varepsilon, \gamma) \ni (q', w),$$

independentemente do símbolo que se encontra sob a cabeça de leitura de input do autómato, e se o valor que se encontra na célula de topo da pilha é γ ,

- não move a cabeça de leitura de input;
- muda para o estado q'
- apaga a célula de topo da pilha (POP);
- coloca, por ordem, cada um dos símbolos de w (PUSH), forçando que cada uma das células já presentes na pilha sejam movidas para a posição imediatamente seguinte.

Se

$$\delta(q, a, \varepsilon) \ni (q', w),$$

com $a \in \Sigma$ e $w \in \Gamma^*$, independentemente do símbolo que se encontra no topo da pilha, se o caracter que se encontra sob a cabeça de input é a ,

- move a cabeça de input uma posição para a direita;
- muda para o estado q' ;
- coloca, por ordem, cada um dos símbolos de w (PUSH), forçando que cada uma das células já presentes na pilha sejam movidas para a posição imediatamente seguinte.

Em qualquer momento da execução do PDA A , podemos descrever a sua configuração pela seguinte palavra de $\Gamma^* \times Q \times \Sigma^*$:

$$z_0 p_1 \cdots p_{\ell-1} p_{\ell} q i_m i_{m+1} \cdots i_{n-1}$$

em que

- q é o estado corrente do autómato,

- $i_m i_{m+1} \cdots i_{n-1}$ é o que resta do input para ser lido, sendo que a cabeça de leitura do input está sobre i_m ,
- $z_0 p_1 \cdots p_{\ell-1} p_\ell$ é o conteúdo da pilha sendo p_ℓ o conteúdo da célula de topo da pilha.

Então, a configuração inicial de A para o input $w = \sigma_0 \sigma_1 \cdots \sigma_{n-1}$, será

$$z_0 q_0 \sigma_0 \sigma_1 \cdots \sigma_{n-1}.$$

Podemos descrever a evolução de A durante a avaliação de um input da seguinte forma, sendo $q, q' \in Q, \sigma \in \Sigma, u \in \Sigma^*, p \in \Gamma$ e $v, w \in \Gamma^*$:

- $(vpq\sigma u) \vdash_A (vwq'u)$, se $\delta(q, \sigma, p) \ni (q', w)$;
- $(vpqu) \vdash_A (vwq'u)$, se $\delta(q, \varepsilon, p) \ni (q', w)$;
- $(vq\sigma u) \vdash_A (vwq'u)$, se $\delta(q, \sigma, \varepsilon) \ni (q', w)$.

Definição

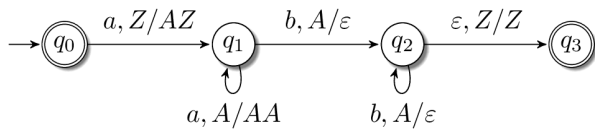
Seja $A = \langle Q, \Sigma, \Gamma, \delta, q_0, z_0, F \rangle$ um autômato de pilha. A linguagem definida por A é o conjunto de palavras para as quais há uma sequência de configurações, de acordo com δ , que começa com uma configuração inicial e termine uma configuração de aceitação. Uma configuração inicial tendo como input a palavra $w \in \Sigma^*$ é uma configuração $z_0 q_0 w$, com q_0 . Uma configuração de aceitação é uma configuração αq_f com $q_f \in F$ e $\alpha \in \Gamma^*$. Portanto a palavra $w \in \mathcal{L}(A)$ se existir

$$z_0 q_0 w \vdash^* \alpha q_f,$$

com $q_f \in F$ e $\alpha \in \Gamma^*$.

Exemplo:

Consideremos a linguagem $L = \{a^n b^n \mid n \in \mathbb{N}\}$. O seguinte autômato de pilha (PDA) A



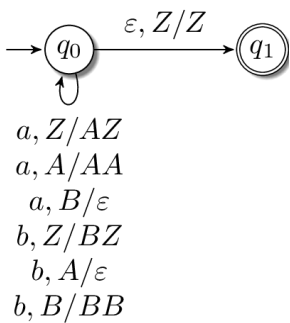
Consideremos a evolução de A , com input $aaabbb$:

$$\begin{aligned} Zq_0 aaabbb \vdash Z A q_1 aabbb \vdash Z A A q_1 abbb \vdash Z A A A q_1 bbb \vdash \\ Z A A q_2 bb \vdash Z A q_2 b \vdash Z q_2 \vdash Z q_3, \end{aligned}$$

portanto, a palavra $aaabbb$ é aceite pelo autômato. Com o input $aabbb$ como não há evolução de A que, com input dessa palavra, partindo de $Zq_0 aabbb$ chegue a uma configuração com estado q_0 ou q_3 e que consuma todo o input. Portanto $aabbb \notin \mathcal{L}(A)$.

Exemplo:

Para a linguagem $\{w \mid w \in \{a, b\}^*, |w|_a = |w|_b\}$ podemos considerar o seguinte autômato de pilha:



Para a palavra *abbaab* temos a seguinte sucessão de configurações:

$$Zq_0abbaab \vdash ZAq_0bbaab \vdash Zq_0baab \vdash ZBq_0aab \vdash \\ Zq_0ab \vdash ZAq_0b \vdash Zq_0 \vdash Zq_1.$$

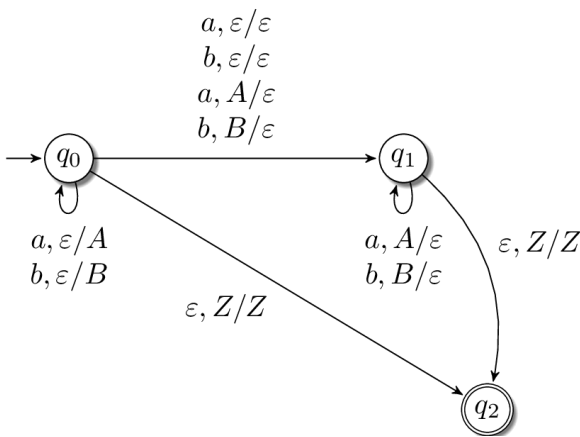
Portanto a palavra é aceite pelo autómato.

Exemplo:

Apesar de alguma “aparência não-determinística”, os dois autómatos anteriores são, na sua essência, determinísticos. Mas para a linguagem dos palíndromos, ou seja $\{w \in \{a, b\}^* \mid w = r^R\}$, não é possível proceder da mesma forma. Uma forma de descrever esta linguagem é

$$\{wxw^R \mid w \in \{a, b\}^*, x \in \{a, b, \varepsilon\}\}.$$

Portanto, necessitamos saber quando deixamos de ler caracteres de w e passamos a ler caracteres de w^R , ou seja qual a posição intermédia da palavra do input. Como lemos o input da esquerda para a direita, não podemos determinar se estamos a ler o caracter na posição intermédia, ou não. O que o autómato tem que fazer é “tentar adivinhar” qual é tal posição intermédia.



Para a palavra *ababa* temos a seguinte sucessão de configurações:

$$Zq_0ababa \vdash ZAq_0baba \vdash ZABq_0aba \vdash ZABq_1ba \vdash ZAq_1a \vdash Zq_1\varepsilon \vdash Zq_2\varepsilon.$$

Para a palavra *abba* temos:

$$Zq_0abba \vdash ZAq_0bba \vdash ZABq_0ba \vdash AZq_1a \vdash Zq_1\varepsilon \vdash Zq_2\varepsilon.$$